



MODULE MIA4 • JOUR 1 / 5

Conception et intégration d'un SGBD NoSQL

Jour 1 — Introduction au NoSQL & MongoDB

Comprendre les principes du Big Data et du NoSQL • Évaluer SQL vs NoSQL • Installer et tester MongoDB • Manipuler les objets via le Shell Mongo

Plan de la journée

7 heures effectives · théorie, démonstrations et atelier pratique

- 1 **Accueil & introduction** — 9h00-9h30 · contexte et objectifs
- 2 **Big Data & principes NoSQL** — 9h30-10h30
- 3 **SQL vs NoSQL, théorème CAP** — 10h45-12h00
-  *Pause déjeuner — 12h00-14h00*
- 4 **Focus MongoDB, JSON & BSON** — 14h00-15h00
- 5 **Installation locale & Docker** — 15h00-16h00
- 6 **Atelier — Shell Mongo & CRUD** — 16h00-17h00

HORAIRES

9h00 → 17h00

 7h effectives

 Pause 10h30-10h45

 Déjeuner 12h-14h

 1 atelier pratique

PRÉREQUIS TECHNIQUES

- ✓ Ordinateur portable
- ✓ Python 3.10+ installé
- ✓ Docker Desktop
- ✓ Connexion internet
- ✓ Bases de SQL et prog.

Objectifs pédagogiques

À l'issue de ce Jour 1, vous serez capables de :



Comprendre le Big Data et le NoSQL

Cerner les 5 V, les limites du relationnel et les cas d'usage



Évaluer SQL vs NoSQL

Identifier avantages, inconvénients et le théorème CAP



Distinguer les 4 familles NoSQL

Clé-valeur, document, colonnes, graphe — savoir choisir



Installer & configurer MongoDB

En local et via Docker — connexion et test



Manipuler les données avec le Shell Mongo

Réaliser les opérations CRUD (Create, Read, Update, Delete) sur des collections et des documents

🚩 **Livrable du jour** : une instance MongoDB fonctionnelle, une collection remplie et 8 commandes CRUD maîtrisées

Big Data : les 5 V et pourquoi le SQL décroche

L'explosion des données a rendu certaines contraintes du relationnel structurellement inadaptées.

V1

Volume

Téraoctets, pétaoctets. Les tables classiques ne tiennent plus sur un seul serveur.

V2

Vélocité

Flux en temps réel : IoT, logs applicatifs, transactions financières.

V3

Variété

Texte, JSON, images, vidéos, logs — le schéma rigide ne suffit plus.

V4

Véracité

Qualité et fiabilité variables des données ; doublons, incohérences, bruit.

V5

Valeur

Extraire de l'information exploitable, sinon la donnée est un coût.

Où le relationnel montre ses limites

✗ Contraintes du SQL sur du Big Data

- **Scalabilité verticale** uniquement (CPU/RAM d'un serveur)
- Schéma **rigide** : chaque évolution = migration
- **Jointures coûteuses** à très gros volume
- Formats semi-structurés (JSON, XML) mal supportés
- **Latence** difficile à garantir sur des flux temps réel

✓ Ce que le NoSQL apporte

- **Scalabilité horizontale** par sharding
- **Schéma flexible** ou schemaless
- **Haute disponibilité** par réplication native
- Formats natifs : JSON, documents imbriqués, graphes
- Latence **stable** à très gros volume

SQL vs NoSQL — comparatif structuré

Critère	SQL — Relationnel	NoSQL
Modèle de données	Tables, lignes, colonnes fixes	Documents, clé-valeur, colonnes larges, graphe
Schéma	Strict, prédéfini (DDL)	Flexible ou schemaless
Langage	SQL standardisé (ANSI)	Propre à chaque moteur (MQL, CQL, Cypher...)
Scalabilité	Verticale (scale-up)	Horizontale (scale-out, sharding)
Transactions	ACID complet	BASE ; ACID partiel (MongoDB 4+ multi-doc)
Jointures	Natives (JOIN)	Rares — dénormalisation privilégiée
Cas d'usage typiques	Compta, ERP, RH, banque	Big Data, IoT, réseaux sociaux, catalogues, cache
Exemples	PostgreSQL, MySQL, Oracle, SQL Server	MongoDB, Redis, Cassandra, Neo4j

💡 **À retenir** : NoSQL n'est pas "meilleur", c'est **différent**. Le bon choix dépend du besoin métier — cohérence forte vs disponibilité, structure vs flexibilité.

Théorème CAP & propriétés BASE

Eric Brewer (2000) : dans un système distribué, sur les 3 propriétés CAP, seules 2 peuvent être garanties simultanément.

C

Chaque paire = un compromis

Le sommet **C**onsistency + **A**vailability signifie tolérer un système non partitionné, etc.

A**P**

Triangle CAP — impossibilité prouvée par Gilbert & Lynch (2002).

Les 3 propriétés

C

Consistency (Cohérence)

Chaque lecture retourne l'écriture la plus récente ou une erreur.

A

Availability (Disponibilité)

Chaque requête reçoit une réponse — même sans garantir la dernière écriture.

P

Partition tolerance (Tolérance au partitionnement)

Le système continue de fonctionner malgré une perte de messages entre nœuds.

CP

MongoDB, HBase, Redis (cluster)

AP

Cassandra, DynamoDB, CouchDB, Riak

CA

SGBDR classiques (mono-nœud) — PostgreSQL, MySQL

Les 4 familles NoSQL

Chaque famille répond à un profil d'usage. Le choix dépend de la nature de la donnée et du type d'accès.



Clé-Valeur

Redis · Memcached · DynamoDB · Riak

Le plus simple : **hash map** distribuée.
Accès $O(1)$ par clé.

Cas : cache, sessions, files d'attente, compteurs temps réel



Document

MongoDB · CouchDB · Firestore · Elasticsearch

Documents **JSON/BSON** imbriqués et flexibles.
Requêtes riches, index secondaires, agrégations.

Cas : catalogues, CMS, profils utilisateurs, back-end WebApp



Colonnes larges

Cassandra · HBase · Bigtable · ScyllaDB

Modèle **columnar** : lignes creuses, colonnes dynamiques.
Écritures massives, agrégations analytiques.

Cas : logs, télémétrie IoT, séries temporelles, analytics



Graphe

Neo4j · JanusGraph · Amazon Neptune · ArangoDB

Nœuds + relations. Traversées **multi-sauts** ultra-rapides.
Langage Cypher (Neo4j).

Cas : réseaux sociaux, recommandation, détection fraude, connaissance

Focus MongoDB — le SGBD document leader

Qu'est-ce que MongoDB ?

SGBD **NoSQL orienté document**, open-source (Server Side Public License), initialement écrit en C++.

Créé en **2007** par la société 10gen (renommée MongoDB Inc.), première version stable en 2009. Cotée au Nasdaq depuis 2017.

Le nom vient de *humongous* — énorme.

Vocabulaire — SQL → MongoDB

Database → **Database**

Table → **Collection**

Row → **Document** (BSON)

Column → **Field**

JOIN → **\$lookup** ou embed

Caractéristiques clés



Documents BSON

Structure JSON binaire, riche, imbriquée, jusqu'à 16 Mo par document



Scalabilité horizontale

Sharding natif, Replica Sets pour haute dispo, auto-failover



Index & agrégation

Index simples, composés, géospatiaux, texte. Aggregation Pipeline puissant



Drivers officiels

Python, PHP, Java, JS/Node, C#, Go, Ruby, Rust, Swift — SDK maintenus

JSON & BSON — les formats de MongoDB

JSON — JavaScript Object Notation

Format **texte**, lisible, universel. Types : string, number, boolean, null, array, object.

Avantages : universel, lisible humain, natif JavaScript.

Limites : pas de type binaire, pas de date, pas de nombres 64-bit, verbeux, lent à parser à grande échelle.

BSON — Binary JSON

Sérialisation **binaire** de JSON, créée par MongoDB.

Types supplémentaires : ObjectId, Date, Binary, Decimal128, Int32, Int64, Regex, Timestamp.

Avantages : parcours rapide, taille prévisible, index efficace.

Exemple : un document utilisateur

```
// document JSON stocké en interne au format BSON
{
  "_id": ObjectId("66f5b9e2c1a4d3e5f6789012"),
  "email": "alice.durand@ipssi.fr",    // String
  "age": 23,                          // Int32
  "tags": ["data", "nosql", "master1"], // Array
  "adresse": { "ville": "Paris", "cp": "75012" }, // Sous-doc
  "cree_le": ISODate("2025-10-15T09:30:00Z") // Date BSON
}
```

Installation locale de MongoDB Community

MongoDB Community Edition est gratuite. Version LTS recommandée : 7.0 ou 8.0.

Windows 10 / 11

1. Télécharger le .msi

mongodb.com/try/download/community

2. Installer en mode **Complete**

Cocher : Install as a Service

3. Installer **MongoDB Compass** (GUI)

4. Vérifier le service

```
net start MongoDB
```

5. Tester la connexion

```
mongosh
```

macOS (Intel / Apple Silicon)

1. Installer **Homebrew** si absent

2. Ajouter le tap officiel

```
brew tap mongodb/brew
```

3. Installer MongoDB Community

```
brew install mongodb-community@7.0
```

4. Démarrer le service

```
brew services start mongodb-community
```

5. Tester

```
mongosh
```

Linux — Ubuntu 22.04+

1. Ajouter la clé GPG

```
curl -fsSL https://pgp.mongodb.com/
server-7.0.asc | sudo gpg -o /usr/share/
keyrings/mongodb.gpg --dearmor
```

2. Ajouter le repo APT et installer

```
sudo apt update
sudo apt install -y mongodb-org
```

3. Démarrer et activer

```
sudo systemctl start mongod
sudo systemctl enable mongod
```

MongoDB via Docker — approche recommandée

Reproductible, isolé, sans dépendance système — c'est l'approche que nous privilégierons pendant le module.

> Option 1 — Lancement rapide (une seule instance)

```
# Lancer un conteneur MongoDB 7.0 avec un volume persistant
docker run -d --name mongo-ipssi \
  -p 27017:27017 \
  -v mongo-data:/data/db \
  -e MONGO_INITDB_ROOT_USERNAME=admin \
  -e MONGO_INITDB_ROOT_PASSWORD=ipssi2025 \
  mongo:7.0
```

🔧 Option 2 — docker-compose (MongoDB + Mongo Express UI)

```
# docker-compose.yml
services:
  mongo:
    image: mongo:7.0
    restart: always
    ports: ["27017:27017"]
    volumes: ["mongo-data:/data/db"]
    environment:
      MONGO_INITDB_ROOT_USERNAME: admin
      MONGO_INITDB_ROOT_PASSWORD: ipssi2025

  mongo-express:
    image: mongo-express:latest
    ports: ["8081:8081"]
    depends_on: [mongo]

volumes:
  mongo-data: {}
```

✅ Bonnes pratiques

- Toujours **monter un volume** — sinon perte des données à chaque restart
- Utiliser **.env** pour les identifiants, jamais en clair
- Épingler la **version majeure** (**mongo:7.0**)
- **Réseau dédié** pour communiquer entre conteneurs
- Sauvegarder régulièrement avec **mongodump**

Le Shell Mongo — **mongosh**

Commandes de navigation essentielles

```
# Connexion à une instance locale
mongosh "mongodb://localhost:27017"

# Connexion authentifiée (Docker)
mongosh -u admin -p ipssi2025 --authenticationDatabase admin

# Lister les bases
> show dbs

# Sélectionner / créer une base (lazy)
> use master1_nosql

# Base courante
> db

# Lister les collections
> show collections

# Créer une collection
> db.createCollection("etudiants")

# Compter les documents
> db.etudiants.countDocuments()

# Aide interactive
> help
> db.etudiants.help()
```

Client officiel MongoDB. Interpréteur JavaScript avec autocomplétion, coloration, historique.

À savoir sur mongosh



JavaScript natif

Vous pouvez écrire des boucles, fonctions, variables — tout le JS moderne (ES2020+) est disponible.



Création paresseuse (lazy)

`use maBase` ne crée rien tant qu'aucun document n'est inséré.



Alternative graphique : MongoDB Compass

Client officiel GUI, gratuit. Visualisation des documents, écriture de requêtes, exploration du schéma, plans d'exécution, index — parfait pour débiter et déboguer.

Opérations CRUD — les 4 piliers

➕ CREATE — insertOne / insertMany

```
db.etudiants.insertOne({
  nom: "Durand", prenom: "Alice",
  age: 23, promo: "M1-DDIA"
})

db.etudiants.insertMany([
  { nom: "Martin", age: 22 },
  { nom: "Nguyen", age: 24 }
])
```

🔍 READ — find / findOne

```
// tous les documents
db.etudiants.find()

// filtre + projection
db.etudiants.find(
  { age: { $gte: 23 } },
  { nom: 1, _id: 0 }
).sort({ age: -1 }).limit(5)
```

✎ UPDATE — updateOne / updateMany

```
db.etudiants.updateOne(
  { nom: "Durand" },
  { $set: { email: "alice@ipssi.fr" },
    $inc: { age: 1 } }
)

db.etudiants.updateMany(
  { promo: "M1-DDIA" },
  { $set: { actif: true } }
)
```

🗑 DELETE — deleteOne / deleteMany

```
db.etudiants.deleteOne(
  { nom: "Martin" }
)

db.etudiants.deleteMany(
  { actif: false }
)

// Vider une collection
db.etudiants.deleteMany({})
```

ATELIER 1H

TP — Installation MongoDB & premières manipulations

☰ Consignes

- 1 **Installer MongoDB** via Docker : lancer le conteneur `mongo:7.0` avec volume persistant et identifiants admin.
- 2 **Se connecter** avec `mongosh` puis avec **MongoDB Compass**. Vérifier que les deux clients voient l'instance.
- 3 **Créer la base** `bibliotheque` et la collection `livres`.
- 4 **Insérer 20 livres** — champs : `titre`, `auteur`, `annee`, `genre`, `note`, `tags[]`.
- 5 **Requêter** les livres postérieurs à 2010, triés par `note` décroissante, en ne renvoyant que `titre` et `auteur`.
- 6 **Modifier** tous les livres du genre "SF" en ajoutant un champ `"favori": true`.
- 7 **Supprimer** les livres dont la note est inférieure à 5, puis compter le total restant.

🚩 LIVRABLES

- Un fichier `tp1.js` contenant toutes vos commandes
- Un **export JSON** de la collection `livres`
- Une capture d'écran **Compass** avec la collection
- Envoi via **Teams** avant 17h

💡 ÉVALUATION

- Cohérence des commandes
- Utilisation de `$set`, `$gte`, `$in`
- Structuration des documents
- Bonus : usage d'un tableau `tags[]`
- Bonus : script `.js` exécuté via `mongosh <file>`

Ce qu'il faut retenir du Jour 1



Le NoSQL n'est pas anti-SQL

C'est un ensemble de moteurs conçus pour les volumes et formats que le relationnel gère mal.



CAP dicte les compromis

MongoDB est CP par défaut. On perd un peu de disponibilité pour garder de la cohérence.



JSON humain · BSON machine

Vous écrivez du JSON, MongoDB stocke et indexe du BSON binaire optimisé.



Docker = reproductibilité

Une image, un volume, on repart sur toute machine. Aucune installation système.



mongosh est du JavaScript

Vous pouvez scripter, boucler, factoriser vos commandes — pensez script `.js`.



CRUD = fondation

Tout ce que vous ferez plus tard (indexation, agrégation, drivers) part de ces 4 opérations.

→ **Demain — Jour 2** : nous passerons au **schéma design** MongoDB, à l'**indexation**, au **profiling** et à l'utilisation des drivers Python et Java.